ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who supported and guided me in the successful completion of my internship.

First and foremost, I express my heartfelt thanks to the University College of Engineering Kancheepuram, for providing me with the opportunity to undertake this internship as part of my academic curriculum.

I extend my sincere gratitude to our respected Dean for providing the necessary support and encouragement. I would also like to thank the Head of the Department, Department of Computer Science and Engineering, for the valuable support and encouragement throughout my internship.

Finally, I express my heartfelt gratitude to my parents, friends, and family members for their constant support, encouragement, and motivation throughout this journey.

I once again extend my sincere thanks to everyone who contributed directly or indirectly to the successful completion of my internship. The internship provided an opportunity to understand software architecture, real-time monitoring, automation, testing and practical system development through the Smart File Monitoring and Auto-Organization System.

I express my sincere thanks to my Faculty Guide for the valuable guidance, constructive suggestions, and continuous encouragement, which helped me successfully complete this internship.

I am deeply grateful to my Instructor for the continuous guidance, patience in clarifying my doubts, and for providing me with valuable technical knowledge and practical experience throughout the internship. I would also like to thank the personnel of the organization where I interned for providing me with the necessary resources and a supportive environment to gain practical exposure.

ABSTRACT

The Smart File Monitoring and Auto-Organization System (FileSentry) is a modular software system designed to monitor configured folders, detect file-system events and automate repetitive file-organization tasks. The system separates monitoring, metadata processing, rule evaluation, action execution and presentation, so that file activity can be observed and managed systematically.

A File Watcher detects events such as file creation, modification, movement and renaming. Each event is represented with file metadata and processed by an automation engine that evaluates predefined rules and performs actions such as moving or renaming files. Duplicate detection using metadata and SHA-256 content signatures, audit logging and a dashboard for monitoring provide visibility into the state and outcome of every automated operation.

The architecture combines a Windows-oriented desktop and client component using operating-system file APIs and C#/.NET with a Python/Flask web dashboard, communicating through REST and JSON, supported by logging and telemetry for observability. The design emphasizes modularity, error handling, testing and future scalability. Future directions include a visual rule editor, cloud-storage integration, cross-platform support and more advanced intelligent organization.

Keywords

File Monitoring, File Organization, Automation, File Watcher, Rule Engine, Duplicate Detection, Dashboard, Audit Logging

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**iv**
	LIST OF FIGURES	**ix**
	LIST OF ABBREVIATIONS	**x**
1.	**INTRODUCTION**	**01**
	1.1 OBJECTIVE	01
	1.2 MOTIVATION	02
	1.3 SCOPE	02
	1.4 LIMITATION	03
2.	**PRELIMINARIES**	**04**
	2.1 WORKING OF THE PROPOSED SYSTEM	04
	2.2 SYSTEM LAYERED PIPELINE	04
	2.3 TECHNOLOGIES USED	04
	2.4 DEPLOYMENT CONSIDERATIONS	05
3.	**LITERATURE SURVEY**	**06**
	3.1 AUTOMATED FILE MANAGEMENT SYSTEMS	06
	3.2 REAL-TIME FILE SYSTEM MONITORING	06
	3.3 RULE-BASED AUTOMATION	06
	3.4 DUPLICATE DETECTION	06
	3.5 FILE METADATA ANALYSIS	06
	3.6 DASHBOARD-BASED MONITORING	07
	3.7 DESKTOP/WEB-BASED FILE MANAGEMENT	07
	3.8 SUMMARY OF LITERATURE THEMES	07

CHAPTER NO.	TITLE	PAGE NO.
4.	**PROPOSED ARCHITECTURE**	**08**
	4.1 INTRODUCTION	08
	4.2 SYSTEM ARCHITECTURE	08
	4.3 SYSTEM OVERVIEW	09
	4.4 SYSTEM WORKFLOW	10
	4.5 SYSTEM SIMULATION	11
	4.6 ADVANTAGES	12
	4.7 DESIGN CONSTRAINTS	12
5.	**SYSTEM DESIGN**	**13**
	5.1 INTRODUCTION	13
	5.2 ENTITY-RELATIONSHIP DIAGRAM	13
	5.3 USE CASE DIAGRAM	15
	5.4 DATA FLOW DIAGRAMS	16
	5.5 DESIGN RATIONALE	17
6.	**SYSTEM IMPLEMENTATION**	**18**
	6.1 METHODOLOGY	18
	6.2 MODULES	18
	6.3 MODULES DESCRIPTION	18
	6.3.1 File Watcher Module	19
	6.3.2 File Scanning and Metadata Module	19
	6.3.3 Automation Engine	19
	6.3.4 Duplicate Detection	20
	6.3.5 Dashboard and API	20
	6.3.6 Logging and Telemetry	20

CHAPTER NO.	TITLE	PAGE NO.
	6.4 ALGORITHMS	20
	6.4.1 Event Detection Algorithm	21
	6.4.2 Rule Evaluation	22
	6.4.3 Duplicate Detection Flow	22
	6.4.4 Action Execution	22
	6.4.5 Concurrency and Synchronization	22
	6.5 COMMUNICATION INTERFACE (API REFERENCE)	23
	6.5.1 Monitoring Services	23
	6.5.2 Rule Services	23
	6.5.3 Logging Services	23
	6.5.4 Dashboard Data	23
	6.5.5 Error Handling	23
7.	**EVALUATION METRICS**	**24**
	7.1 TESTING APPROACH	24
	7.2 PERFORMANCE AND OBSERVABILITY	24
	7.3 INTEGRATION VALIDATION	24
	7.4 RESULT INTERPRETATION	24
8.	**SYSTEM REQUIREMENTS**	**25**
	8.1 SOFTWARE REQUIREMENTS	25
	8.2 HARDWARE REQUIREMENTS	25
	8.3 NETWORK AND STORAGE CONSIDERATIONS	25

9. RESULTS AND DISCUSSIONS 26

9.1 INTRODUCTION 26

9.2 FILESENTRY USER PORTAL 26

9.2.1 Login 27

9.2.2 Main Dashboard 28

9.2.3 Monitored Folders 29

9.2.4 File Activity 30

9.2.5 Automation Rules 31

9.2.6 Rule Editor 32

9.2.7 Duplicate Detection 33

9.2.8 Analytics 34

9.2.9 Activity Logs 35

9.2.10 System Settings 36

10. CONCLUSION AND FUTURE WORK 37

10.1 CONCLUSION 37

10.2 FUTURE ENHANCEMENT 37

REFERENCES 38

APPENDIX 39

LIST OF FIGURES

FIGURE	NAME OF THE FIGURE	PAGE NO.
4.1	Smart File Monitoring and Auto-Organization System, System Architecture	09
4.2	Smart File Monitoring and Auto-Organization System, End-to-End Workflow	10
5.1	FileSentry, Conceptual Data Model	13
5.2	FileSentry, Use Case Diagram	15
5.3	FileSentry, Data Flow Diagram	16
6.1	FileSentry, Automation / Rule Processing Flow	21
9.1	Login / Access Screen	27
9.2	Main Dashboard, Monitoring Overview	28
9.3	Monitored Folders, Watcher Configuration	29
9.4	File Activity, Recent Events	30
9.5	Automation Rules, Rule Status	31
9.6	Automation Rule Editor, Conditions and Actions	32
9.7	Duplicate Detection, Candidate Files	33
9.8	Analytics, Monitoring and Automation Trends	34
9.9	Activity Logs, Audit History	35
9.10	System Settings, Monitoring Configuration	36

LIST OF ABBREVIATIONS

ABBREVIATION	FULL FORM
API	Application Programming Interface
REST	Representational State Transfer
JSON	JavaScript Object Notation
UI	User Interface
OS	Operating System
DB	Database
DFD	Data Flow Diagram
ER	Entity-Relationship
HTTP	Hypertext Transfer Protocol
SHA-256	Secure Hash Algorithm, 256-bit
CPU	Central Processing Unit
RAM	Random Access Memory
I/O	Input / Output
UAT	User Acceptance Testing
SDK	Software Development Kit
CI	Continuous Integration

CHAPTER 1
INTRODUCTION

Files accumulate continuously in documents, downloads, reports, images and temporary folders, and organizing them by hand is repetitive, error-prone and difficult to audit. Users and teams rarely know what changed in a folder, when it changed, or why a file ended up where it did. The Smart File Monitoring and Auto-Organization System, referred to throughout this report by its product name FileSentry, addresses this problem by observing configured folders, detecting file-system events as they occur and applying rule-based automation to organize files, while recording every action in an auditable activity trail.

The remainder of this report is organised as follows. Chapter 2 describes the working of the system, its layered pipeline and the technologies used. Chapter 3 reviews the literature themes that motivate the design. Chapter 4 presents the proposed architecture, and Chapter 5 the system design through the conceptual data model, use cases and data-flow diagrams. Chapter 6 describes the implementation, covering the modules, algorithms and communication interface. Chapters 7 and 8 present the evaluation approach and system requirements, Chapter 9 the user interface, and Chapter 10 the conclusion and future work.

Most simple file utilities either run periodic scans or apply a fixed set of hidden behaviours, which leaves the user unable to explain an automated action after the fact. FileSentry instead separates monitoring, metadata processing, rule evaluation, action execution and presentation into distinct modules, so that each stage can be understood, tested and extended independently. Its scope covers monitored-folder configuration, file event detection, metadata processing, automation rules, organization actions, duplicate detection, logging and dashboard-based monitoring. Several directions, including cloud integration and cross-platform deployment, are treated as future enhancements.

1.1 OBJECTIVE

FileSentry is designed to reduce the manual effort involved in monitoring, classifying and organizing files. The system observes configured folders, detects file-system activity, analyses relevant metadata and applies automation rules to perform repeatable actions such as moving or renaming files. The objective is to provide a structured monitoring and organization workflow in which file activity can be detected, processed, acted upon and recorded.

The specific objectives are to detect file creation, modification, movement and renaming in real time through operating-system notifications; to represent each event as a structured FileEvent carrying path, size, timestamps and attributes; to evaluate explicit, prioritized rules against each event; to identify duplicate candidates for review using metadata and SHA-256 content signatures; to record every outcome in an audit log; and to present monitoring state through a dashboard communicating over REST and JSON.

1.2 MOTIVATION

Modern users and teams accumulate documents, images, reports, downloads and temporary files across multiple folders. Manual organization becomes repetitive and makes it difficult to understand what changed and when. FileSentry addresses this through continuous monitoring and rule-driven automation, so that organization policy is expressed once and applied consistently.

A second motivation is accountability. Automation that acts silently on a user's files is only acceptable if its behaviour can be inspected. FileSentry therefore pairs every automated action with an audit entry and favours explicit rules over inferred behaviour, so that the reason for any file operation can be traced to a specific event and a specific rule rather than remaining invisible.

1.3 SCOPE

The project covers monitored-folder configuration, file event detection, metadata processing, automation rules, organization actions, duplicate detection, logging, dashboard-based monitoring and reporting-oriented views. The architecture is organised as five layers: presentation, API and communication, monitoring and business logic, data access, and system and storage. It combines a Windows-oriented monitoring component written for the operating-system file APIs with a web-based dashboard exchanging JSON over REST.

The architecture is deliberately modular so that future versions can introduce more advanced classification, cloud storage integration, cross-platform support and learning from user behaviour without replacing the existing detection, rule and logging components.

Within this scope, a user can register folders for monitoring, observe the resulting file events, define prioritized rules that move or rename files, review duplicate candidates before any destructive action, and inspect the complete audit trail. Operational visibility, including system status, analytics and diagnostic logs, is provided through the dashboard.

1.4 LIMITATION

- Cloud-storage integrations (Google Drive, Dropbox and OneDrive), distributed processing, cross-platform Electron deployment and natural-language or behaviour-learning functionality are proposed extensions and are not part of the current implementation.
- The visual drag-and-drop rule editor is a planned dashboard enhancement; the rule editor screen in Chapter 9 is a representative mockup of the intended feature.
- The watcher relies on Windows file-system notification APIs, which ties the current implementation to a Windows deployment target.

CHAPTER 2
PRELIMINARIES

2.1 WORKING OF THE PROPOSED SYSTEM

FileSentry operates as a monitoring and automation system. A configured folder is watched for file-system changes. When an event occurs, the monitoring component captures it and passes file information to downstream processing. Metadata is analysed, applicable rules are evaluated, and an action such as a move or rename can be executed. The event and its outcome are then recorded so that the dashboard and logs can present an auditable history of what the system did and why.

Detection, analysis, decision, action and logging are kept as separate stages. The watcher never performs organization logic itself; it hands the affected path and event type to the metadata processor, which builds a structured FileEvent for the automation engine.

Error handling is part of the normal flow rather than an exception to it. If a file is locked, a path becomes inaccessible or an action fails, the failure is recorded in the log and monitoring continues, so that the audit trail remains complete and a single failing file cannot interrupt the observation of other folders.

2.2 SYSTEM LAYERED PIPELINE

The pipeline is organised as five layers. Layer 1, Presentation, exposes monitoring and configuration views through the dashboard and desktop client. Layer 2, API and Communication, exchanges structured JSON data over RESTful interfaces. Layer 3, Monitoring and Business Logic, detects events, evaluates rules, executes actions and performs duplicate detection. Layer 4, Data Access, handles metadata, rules, events and audit records. Layer 5, System and Storage, interacts with the Windows file system, the persistent database and the log store.

2.3 TECHNOLOGIES USED

FileSentry uses Windows file-system APIs for event notification, with C#/.NET for the desktop and client components that sit close to the operating system. Python and Flask provide the web dashboard, where REST endpoints and JSON serialization are the priority. JSON is the interchange format because it is natively supported by Flask, simple to parse from .NET and readable during debugging.

PostgreSQL is identified as the persistent store for metadata, rules and audit records, because its transactional guarantees matter when several rules may act on the same file. Observability is addressed through detailed logging with Elasticsearch and Kibana for centralized analysis, and Prometheus and Grafana for performance monitoring. API behaviour is validated with Postman.

This division reflects a common pattern in systems that combine an operating-system-facing agent with a browser-accessible management layer: the agent is written close to the platform it observes, while the dashboard is written for portability and ease of iteration. The same boundary is what allows the dashboard to be re-implemented, for example as a native desktop panel, without changes to the watcher or rule-evaluation logic.

2.4 DEPLOYMENT CONSIDERATIONS

The layered architecture supports more than one deployment arrangement. In the simplest case the watcher, automation engine, dashboard and database run on a single machine, which suits an individual user monitoring local folders. In an organizational setting the dashboard and database can run on a central server while lightweight watcher agents run on each monitored machine and report events over the network.

CHAPTER 3

LITERATURE SURVEY

3.1 AUTOMATED FILE MANAGEMENT SYSTEMS

Automated file management systems reduce the manual burden of sorting, renaming and archiving files by expressing organization policy as software logic rather than repeated manual action. FileSentry follows this direction by combining a monitoring layer with a configurable automation engine, so that organization behaviour is defined once and applied consistently across a monitored folder structure.

3.2 REAL-TIME FILE SYSTEM MONITORING

File monitoring systems must detect changes with low delay while avoiding continuous polling. FileSentry relies on direct Windows API notification mechanisms and a dedicated watcher component, so that file creation, modification, movement and renaming trigger downstream processing immediately and CPU and I/O overhead stay low compared with repeatedly scanning a directory tree.

3.3 RULE-BASED AUTOMATION

Rule-based automation is appropriate when users can express predictable organization policies. A rule connects file characteristics or events with an action. This provides traceability, because the reason for an automated action is represented explicitly rather than hidden inside an opaque model, and it keeps behaviour predictable and testable.

3.4 DUPLICATE DETECTION

Duplicate detection reduces unnecessary storage and helps users identify repeated files. Metadata comparison alone can produce false positives and false negatives, since unrelated files may share a name or size while a renamed copy would

not match. FileSentry therefore also uses content-derived signatures such as SHA-256, and surfaces candidates for review rather than deleting files automatically.

3.5 FILE METADATA ANALYSIS

File path, size, timestamps and attributes provide useful signals for organization. FileSentry represents each event as a FileEvent containing the file path, size, modification time, last access time and file attributes, making metadata processing a central part of the architecture and allowing rules to evaluate conditions such as file age or size without re-querying the file system.

3.6 DASHBOARD-BASED MONITORING

A dashboard gives users a human-readable view of automated activity. FileSentry includes a web-based dashboard intended to move from passive logging toward active management, visual rules and reporting. Logging, API testing, automated testing and performance monitoring form an observability strategy that supports regression control as the components evolve.

3.7 DESKTOP/WEB-BASED FILE MANAGEMENT

Many file-organization tools operate purely as desktop utilities with no browser-accessible view of their activity. FileSentry separates the operating-system-facing watcher from a web-based dashboard communicating over REST and JSON. This lets the dashboard be accessed independently of the monitored machine and keeps the presentation layer decoupled from low-level file-system code.

3.8 SUMMARY OF LITERATURE THEMES

These themes map onto specific modules: automated file management and rule-based automation to the Automation Engine, real-time monitoring to the File Watcher, metadata analysis to the Metadata Processor.

CHAPTER 4

PROPOSED ARCHITECTURE

4.1 INTRODUCTION

The proposed architecture separates monitoring, processing, automation, communication and presentation responsibilities. This modularity is intended to improve maintainability and provide a foundation for future scaling. The monitoring component and the web dashboard are independent parts of the system that exchange structured JSON, so that either side can be tested, redeployed or replaced without rebuilding the other. A failure or restart in one component, for example redeploying the dashboard, does not require the watcher to stop observing folders.

This separation of concerns is what allows the five layers described in the following sections to be developed, tested and evolved largely independently of one another. Each layer exposes a narrow contract to the layer above it, which keeps operating-system interaction, business rules and presentation from becoming entangled.

The architecture also keeps the system easy to reason about during development and troubleshooting. Because detection, analysis, decision, action and logging are distinct stages, a fault can be localised to a single stage by examining the corresponding log entries, and a stage can be replaced or extended, for example with a new classification mechanism, without disturbing the others.

4.2 SYSTEM ARCHITECTURE

Figure 4.1 presents the FileSentry architecture. A user interacts with the dashboard or client in the Presentation layer. Requests and monitoring data travel through the API and Communication layer as JSON over RESTful interfaces. The Monitoring and Business Logic layer contains the File Watcher, the metadata

processor, the automation engine and duplicate detection. These components observe configured folders, produce file events, extract metadata and evaluate rules.

Actions are executed against the file system, while the Data Access layer maintains file metadata, rules, events and audit records in the System and Storage layer, which comprises the Windows file system, the PostgreSQL database and the log store. Logging and telemetry cut across all layers so that every stage leaves an inspectable trace.

The five layers each have a defined responsibility. The Presentation layer provides the dashboard and desktop client views for monitoring and configuration. The API and Communication layer carries structured JSON between the client and the dashboard service. The Monitoring and Business Logic layer is where all decision-making occurs: the watcher, the metadata processor, the automation engine and duplicate detection. The Data Access layer isolates how metadata, rules, events and audit records are stored and retrieved, and the System and Storage layer provides the Windows file system, the database and the log store on which everything else depends.

Placing duplicate detection in the business logic layer rather than the data layer is a deliberate choice: although it depends on stored metadata, it is a decision-making activity that compares candidates and groups them for review, and so belongs with rule evaluation rather than with storage.

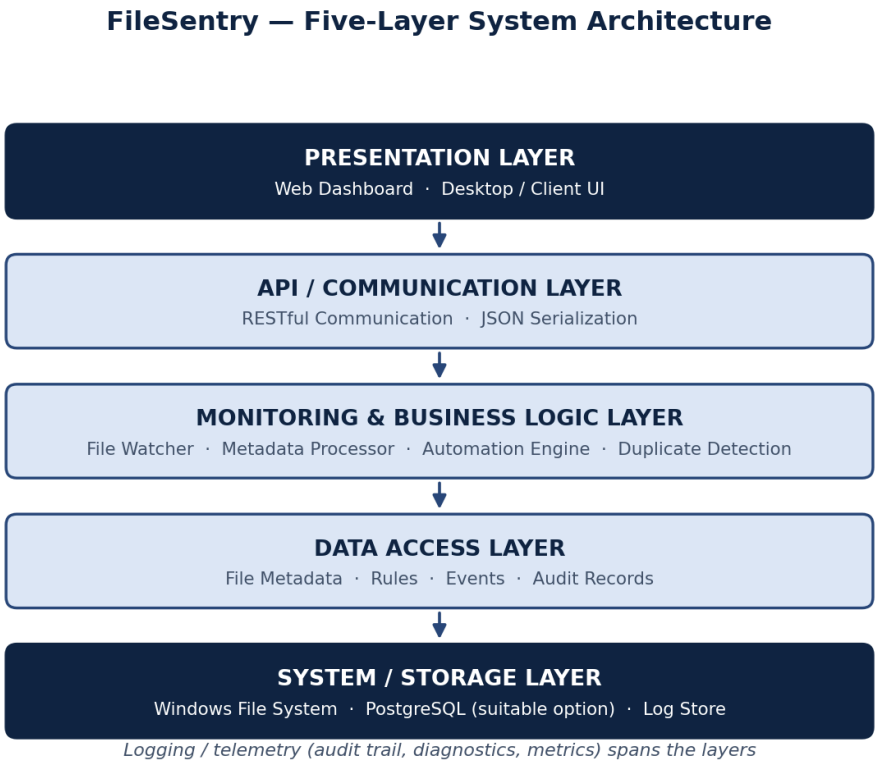


Figure 4.1: Smart File Monitoring and Auto-Organization System, System Architecture

4.3 SYSTEM OVERVIEW

The system begins with folder configuration and monitoring. Events such as file creation, rename, move and modification are detected by the File Watcher through Windows file-system notifications. Each event is transformed into a structured FileEvent containing the path, size, modification time, last access time and attributes of the affected file, and is sent through the processing pipeline.

The rule engine then evaluates the applicable conditions in priority order before an action is performed. Duplicate detection runs as a separate, separable stage so that content hashing over large files can be scheduled or throttled independently of real-time event handling. The outcome of every stage is written to the audit trail and exposed through the dashboard for review.

The dashboard presents monitoring state, configured folders, file activity, automation rules, duplicate candidates, analytics, activity logs and system settings. It

consumes structured JSON summaries of already-processed state and never touches the file system directly, which keeps the presentation layer simple and independent of operating-system details.

A prioritized event queue sits between detection and rule evaluation, so that bursts of events, for example when an archive is extracted into a monitored folder, are absorbed without blocking the watcher. Time-sensitive events are processed ahead of background activity such as duplicate scanning, and synchronization mechanisms protect shared state when more than one rule could act on the same file.

4.4 SYSTEM WORKFLOW

Figure 4.2 represents the end-to-end flow. The sequence deliberately separates detection, analysis, decision, action and logging so that each stage can be tested and extended independently. The core principle of the workflow is: detect, understand, decide, act, record and display.

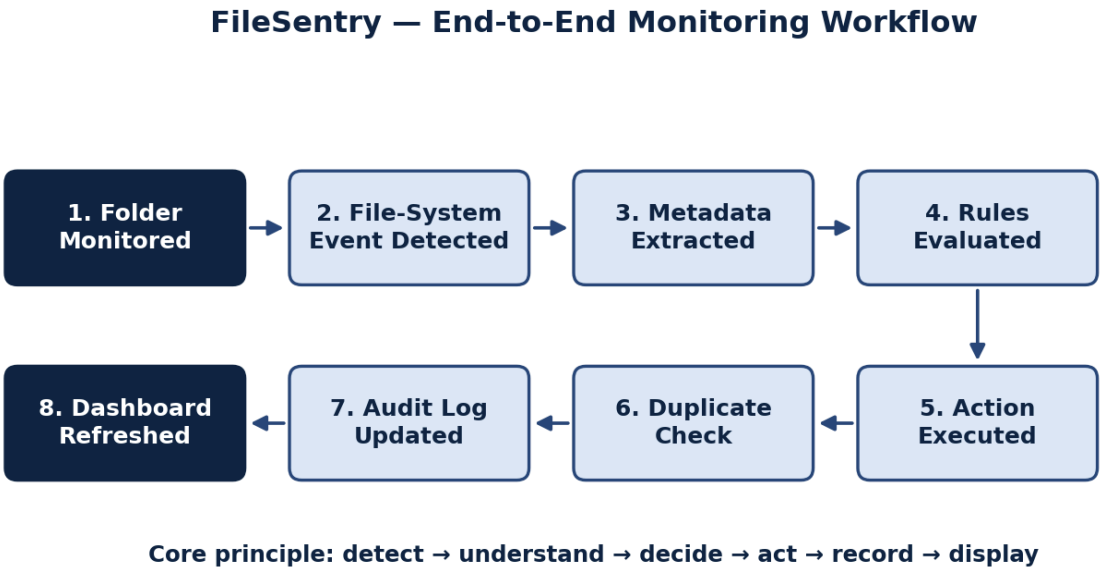


Figure 4.2: Smart File Monitoring and Auto-Organization System, End-to-End Workflow

The workflow starts when a folder is registered for monitoring. When a file-system event is detected, its metadata is extracted and the enabled rules are

evaluated. The selected action is executed, a duplicate check is performed, the audit log is updated and the dashboard view is refreshed. Failures at any stage are recorded in the log rather than terminating monitoring, so that a single locked or inaccessible file cannot interrupt observation of the remaining folders.

Each stage of the workflow produces a well-defined output that the next stage consumes. Detection yields a path and event type; metadata extraction yields a FileEvent; rule evaluation yields the selected actions; execution yields a change in file-system state; and logging yields an audit entry that the dashboard then presents. Because every stage records its outcome, the workflow can be audited end to end, and a test can introduce a deliberate failure at any stage and confirm that it appears as a log entry rather than propagating silently.

4.5 SYSTEM SIMULATION

A typical simulation starts when a new file enters a monitored folder. The File Watcher detects the creation event and identifies the path and event type. The metadata processor collects the size, timestamps and attributes and builds a FileEvent. The automation engine identifies the matching rules by priority, and the selected organization action, such as moving the file to a category folder or renaming it, is executed.

The duplicate-detection stage then checks whether the new file is a candidate duplicate of a known file, using metadata and, where appropriate, a SHA-256 signature. Finally the result is written to the audit trail and the dashboard exposes the event for review. Each step is observable in the File Activity and Activity Logs views, which allows the whole sequence to be reconstructed after the fact.

A second scenario illustrates review-first duplicate handling. When a file whose content already exists elsewhere in a monitored folder is copied in, the metadata comparison flags it as a candidate and the SHA-256 digest confirms that its content

matches a known file. The candidate is grouped with the existing file and surfaced in the Duplicate Detection view together with the basis for the match, and no file is moved or removed until the operator has reviewed it. The scenario shows how automation and human oversight are combined for an operation that is otherwise difficult to reverse.

4.6 ADVANTAGES

- Modular development: the watcher, metadata processor, automation engine, duplicate detection and dashboard can be developed and tested independently.
- Real-time monitoring: operating-system notifications provide low-latency detection without continuous polling.
- Rule traceability: every automated action can be traced to a specific event and rule, rather than to an opaque model.
- Centralized logging: the audit trail and diagnostic logs give a single, inspectable record of system behaviour.
- Future scalability: additional integrations and more sophisticated classification can be introduced without replacing the entire system.

4.7 DESIGN CONSTRAINTS

The five-layer design is shaped by several constraints. The most significant is the reliance on Windows-specific file-system notification APIs for the watcher module, which ties the current implementation to a Windows deployment target until the cross-platform Electron direction listed under future enhancements is pursued.

A second constraint is the explicit preference for traceable, rule-based automation over inferred or learned behaviour. This is a deliberate trade-off: a rule-based engine is more predictable and auditable, at the cost of requiring the user to

state the organization policy explicitly. A third constraint concerns duplicate handling. Because hashing large files and folders is computationally expensive, duplicate detection is a separable pipeline stage that can be scheduled independently of real-time event processing.

Taken together, these constraints explain why the architecture favours predictability and auditability over breadth: it is easier to extend a transparent, modular system with new capabilities than to make an opaque one explainable after the fact.

Another important consideration is the coordination between the individual layers of the architecture. Each layer must perform its assigned responsibilities while exchanging information with the other layers in a consistent manner. For example, file-system events generated by the watcher must be processed carefully before they are passed to the rule engine and subsequent organization components. This separation helps prevent unnecessary coupling between modules and makes it easier to identify errors during development, testing, and maintenance. It also allows individual components to be modified or enhanced without requiring major changes throughout the entire system.

Performance and reliability are additional considerations in the design. A file-monitoring system may encounter a large number of events when several files are created, modified, renamed, or moved within a short period. Processing every event independently without suitable coordination could increase resource consumption or cause repeated operations. Therefore, the architecture should support controlled event processing, appropriate error handling, and logging mechanisms to maintain reliable operation. Operations that involve significant processing, such as duplicate detection, should remain independent of time-sensitive monitoring tasks so that they do not unnecessarily delay the handling of new file-system events

CHAPTER 5

SYSTEM DESIGN

5.1 INTRODUCTION

This chapter presents the structural and behavioural design of FileSentry: the entities the system maintains and how they relate, the interactions a user can initiate, and the flow of data between the user, the application and the file system. The conceptual model identifies users, monitored folders, file events, file metadata, automation rules and audit records. The use-case and data-flow views describe how these components interact.

5.2 ENTITY-RELATIONSHIP DIAGRAM

Figure 5.1 shows a conceptual data model for FileSentry. A user can configure monitored folders. Each monitored folder can generate multiple file events. Events are associated with file metadata and may be processed by applicable automation rules. Results are retained in audit logs for traceability. The model is conceptual: it identifies the information the system must maintain without prescribing a particular table layout.

Keeping events, metadata, rules and audit records as separate entities allows the rule set to be changed without affecting stored events, and allows the audit trail to retain the outcome of every action even after a rule has been edited or disabled.

The relationship between automation rules and file events is important because a single rule may be applicable to multiple events, while an individual event may be evaluated against more than one rule. Keeping these concepts logically separate avoids embedding organization policies directly into event records. As a result, changes to a rule's conditions or enabled status can be managed independently of the historical events already recorded by the system.

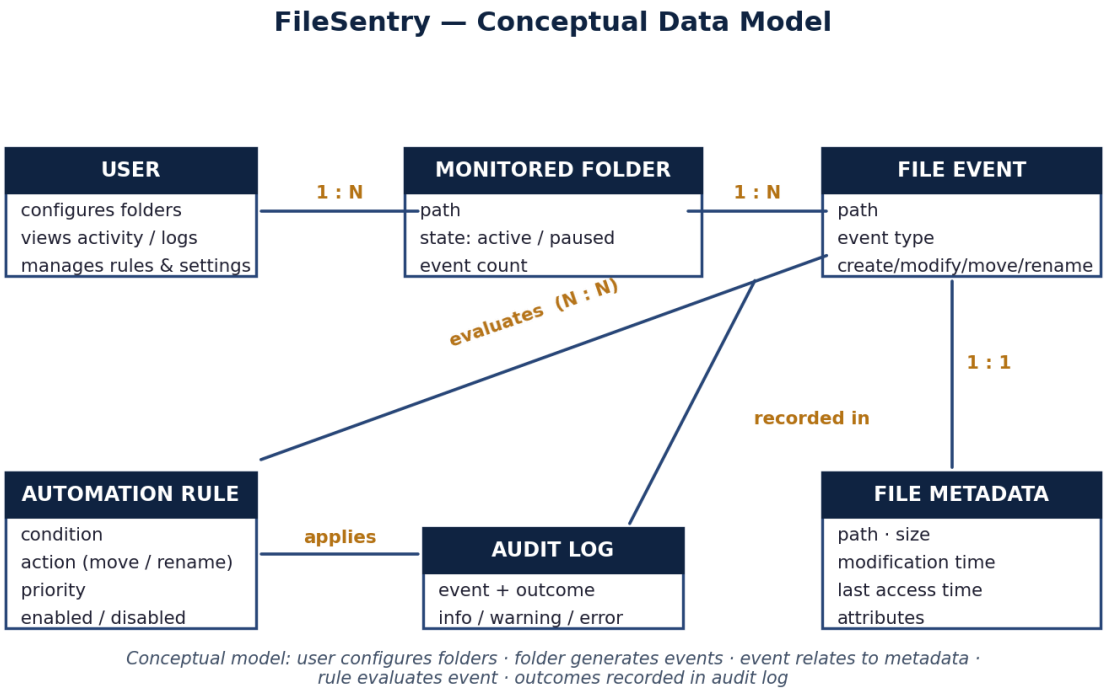


Figure 5.1: FileSentry, Conceptual Data Model

The User entity represents the operator who configures monitored folders and manages rules and settings. A MonitoredFolder records a watched location and its state, active or paused, together with the events it has generated. A FileEvent records the path and the type of change, and relates to a FileMetadata record holding the size, modification time, last access time and attributes. An AutomationRule holds a condition, an action, a priority and an enabled state, and an AuditLog entry records the outcome of each processed event, including informational messages, warnings and errors.

The entities and their relationships can be summarised as follows. The User holds the operator's identity and is related to the MonitoredFolder entities that the operator configures. A MonitoredFolder holds a path, an active or paused state and an event count, and is related to the FileEvent records it generates. A FileEvent holds the path and the event type (create, modify, move or rename) and is related to its FileMetadata record and to the AutomationRule entities that evaluate it. FileMetadata holds the size, modification time, last access time and attributes, and is related to

exactly one FileEvent. An AutomationRule holds a condition, an action, a priority and an enabled state, and is related to the FileEvent records it evaluates. An AuditLog holds the outcome of each event together with its informational, warning and error entries, and is related to both the FileEvent and the AutomationRule that produced the outcome.

5.3 USE CASE DIAGRAM

Figure 5.2 presents the principal user interactions: monitoring folders, viewing file activity, creating automation rules, organizing files, detecting duplicates, reviewing audit logs, viewing analytics and managing settings. A single actor, the user, exercises all of these use cases through the dashboard.

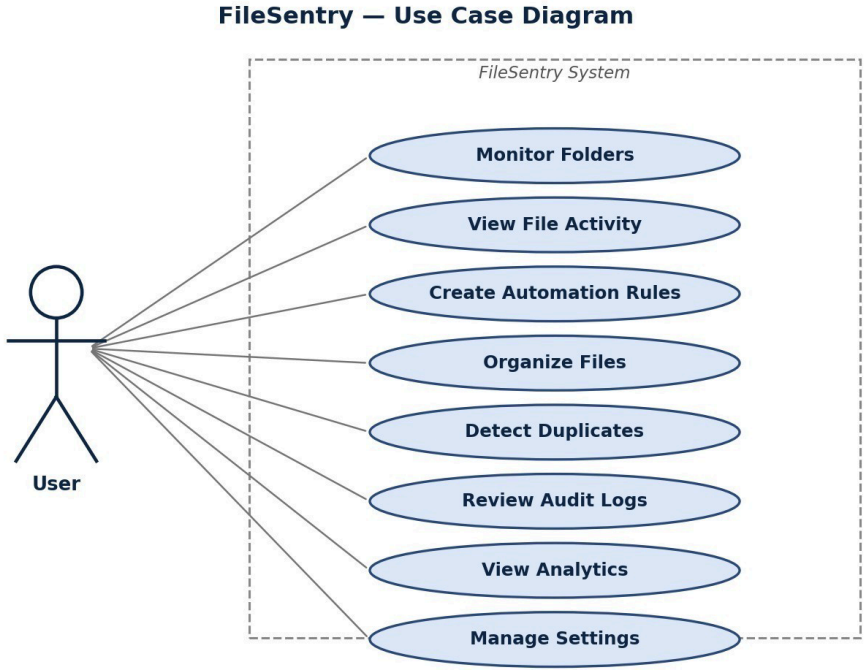


Figure 5.2: FileSentry, Use Case Diagram

Monitor Folders and Manage Settings cover configuration of what is watched and how the system behaves. View File Activity, View Analytics and Review Audit Logs cover observation of what has happened. Create Automation Rules and Organize Files cover the definition and execution of organization policy, and Detect Duplicates

covers identification of repeated files for review. Organizing files and detecting duplicates are system behaviours that the user configures and then reviews, rather than operations the user performs manually on every file.

5.4 DATA FLOW DIAGRAMS

Figure 5.3 presents Level 0 and Level 1 views. At Level 0, the user interacts with FileSentry while the system exchanges data with the file system and persistent records. At Level 1, the process is decomposed into six major processes: event detection, metadata processing, rule evaluation, action execution, logging and dashboard updates.

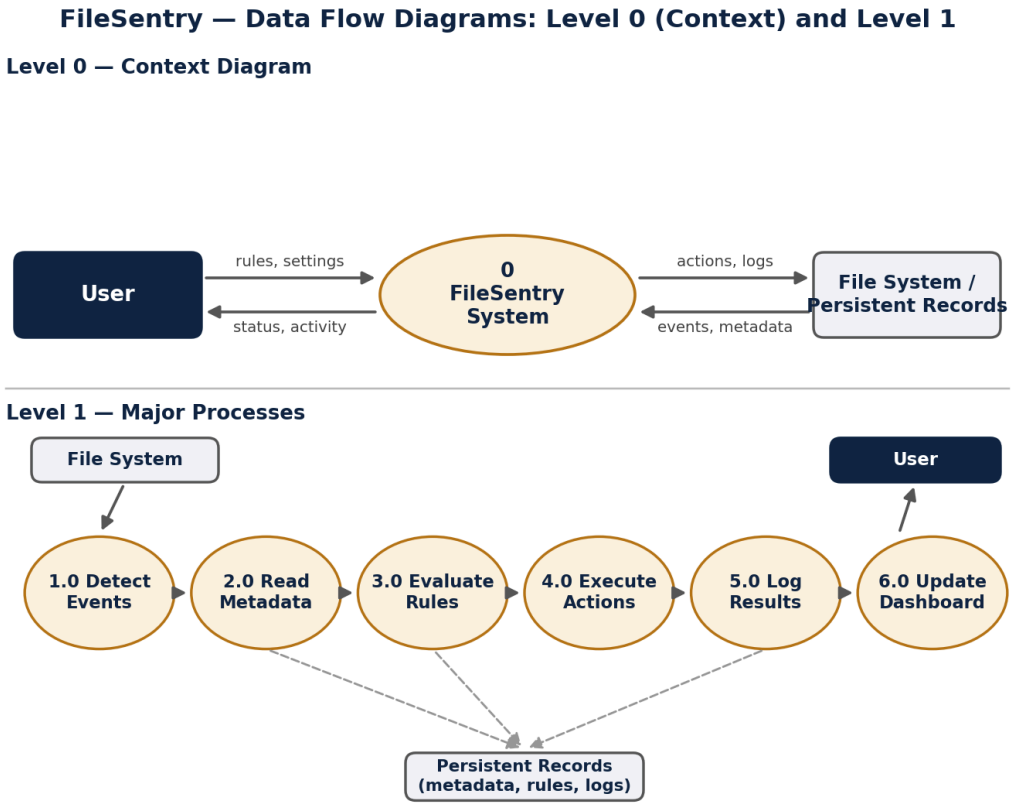


Figure 5.3: FileSentry, Data Flow Diagram

In the Level 1 view, Process 1.0 detects events from the file system. Process 2.0 reads metadata and builds the FileEvent. Process 3.0 evaluates the enabled rules, and

Process 4.0 executes the selected actions against the file system. Process 5.0 records results in persistent records, and Process 6.0 updates the dashboard that the user sees. Separating Process 3.0 from Process 4.0 means that rule evaluation can be tested without touching real files, and that a failed action can be logged and investigated without re-evaluating the rules.

At Level 0, rules and settings flow from the user into the system, and monitoring status and activity flow back to the user. Between the system and the file system and persistent records, events and metadata flow in, while actions and log entries flow out. Treating the entire application as one process at this level makes the external interfaces explicit before the internal decomposition is considered.

5.5 DESIGN RATIONALE

Each view describes the same system from a different angle. The conceptual data model is most useful when reasoning about persistence and the data-access layer. The use-case diagram frames FileSentry purely in terms of what a user can do, which suits non-technical stakeholders. The data-flow diagrams sit between the two, allowing a developer to trace how data moves through the six major processes without committing to their internal implementation.

Taken together, the views justify the module boundaries of Chapter 6: the File Watcher, Metadata Processor, Automation Engine, Duplicate Detection, Dashboard and API, and Logging and Telemetry modules correspond closely to the Level 1 processes, which in turn correspond to the entities and use cases in Figures 5.1 and 5.2.

This consistency across views is a useful check on the design itself: where a use case has no corresponding data-flow process, or a process has no corresponding entity, the design would need further review. The three views were therefore developed together and cross-checked against the module list.

CHAPTER 6

SYSTEM IMPLEMENTATION

6.1 METHODOLOGY

The system is built using a phased development approach. Development begins with requirements and architecture, establishing the layers and the FileEvent representation that every other component depends on. The monitoring service is implemented next, followed by the processing and automation logic, then dashboard communication, logging, testing and deployment-oriented refinement. The layered approach keeps operating-system interaction separate from business rules and presentation, so that each phase builds on a stable lower layer.

In practice this meant defining the FileEvent representation first, because the watcher, metadata processor, automation engine, duplicate detection and logging modules all exchange it. Each module was then specified in terms of its primary input and output, as described in Section 6.2, so that modules could be implemented and tested against synthetic events before being connected to a live file system.

6.2 MODULES

The major modules are the File Watcher, File Scanner and Metadata Processor, Automation Engine, Rule Management, Duplicate Detection, Dashboard and API communication, Logging and Telemetry, and supporting configuration and deployment components. The primary input and output of each module are described in the following paragraph.

The File Watcher takes operating-system file-system notifications as input and produces a raw change event carrying the path and the event type. The Metadata Processor takes that raw change event and produces a structured FileEvent record. The Automation Engine takes a FileEvent together with the current rule set and produces

the selected action or actions. Duplicate Detection takes a FileEvent together with a hash index and produces a duplicate candidate group. The Dashboard and API module takes the stored state, consisting of events, rules and logs, and produces JSON responses for the user interface. Logging and Telemetry takes the processing outcomes of the other modules and produces audit log entries and metrics.

6.3 MODULES DESCRIPTION

The File Watcher observes configured folders and detects changes. The metadata processor converts file-system information into structured FileEvent records. The automation engine evaluates conditions and executes actions. The dashboard presents monitoring information and configuration controls. Logging records processing outcomes and diagnostic information.

6.3.1 File Watcher Module

The watcher is the foundation of real-time monitoring. It uses Windows API notifications for file creation, renaming, movement and modification, allowing the system to respond to file activity without relying only on periodic scans. Notification-based detection minimizes the latency between a change and FileSentry's awareness of it, and reduces CPU and I/O overhead compared with repeatedly walking a directory tree. The watcher is a thin, event-driven layer that hands all further processing to the metadata and automation components.

Because the watcher is the first component to observe a change, the correctness of every downstream stage depends on it reliably capturing the affected path and event type. Building this layer directly on Windows API notifications, rather than a higher-level cross-platform abstraction, reflects a preference for dependable low-latency detection over early portability, which is deferred to the Electron-based direction listed under future enhancements.

6.3.2 File Scanning and Metadata Module

The scanner traverses configured directories and builds file representations containing path, size, modification time, access time and attributes. This normalizes raw operating-system information into the FileEvent representation used throughout the architecture. Capturing size and timestamps when an event is detected lets later stages evaluate conditions such as file age or size without re-querying the file system, and reduces the chance of acting on stale information.

6.3.3 Automation Engine

The automation engine evaluates predefined rules and applies the corresponding actions. Each rule combines a condition over file characteristics, an action, a priority and an enabled or disabled state. Because the engine consumes a generic FileEvent rather than a raw Windows callback, organization policy can change without changing the monitoring layer, and the same engine could in future be driven by other event sources. Matching is centralized in the engine, which makes rule behaviour predictable and testable in isolation.

The engine's responsibility can be summarised as matching a FileEvent against the currently enabled rule set, ordered by priority, and forwarding the matching rule's action, either the first or all applicable depending on configuration, to the execution stage. Keeping this logic out of the watcher and the dashboard is what makes rule behaviour predictable and testable in isolation.

6.3.4 Duplicate Detection

Duplicate detection identifies files that represent repeated content. The module combines metadata comparison with content hashing such as SHA-256, which produces a fixed-length digest of the file's bytes so that identical content yields the same hash regardless of name or location. Candidates are surfaced for review rather

than deleted automatically, which keeps a human decision point for an operation that is otherwise difficult to reverse.

6.3.5 Dashboard and API

The architecture uses RESTful communication and JSON serialization between components. The dashboard provides a visual representation of monitoring state, events, rules and reports. Because the dashboard and the monitoring core communicate only through this boundary, the two sides can be developed, tested and deployed independently, provided the same API contract is honoured.

6.3.6 Logging and Telemetry

Detailed logging records what the system did and why, and is surfaced through the Activity Logs view. Elasticsearch and Kibana support centralized log analysis, while Prometheus and Grafana support performance monitoring of CPU, memory, disk I/O and network utilization. Logging supports auditing of individual actions, whereas telemetry supports capacity planning and early detection of degraded performance.

Centralizing logs through an Elasticsearch and Kibana pipeline allows log data from several monitored machines to be searched and correlated in one place, which becomes increasingly useful as the distributed-processing direction is explored. Until then, the logging module fulfils its core purpose locally: every automated action is explained by a corresponding, inspectable log entry.

6.4 ALGORITHMS

Core processing follows an event-driven pipeline: detect the event, construct the FileEvent, inspect metadata, evaluate rules, execute the selected action, record the result and update the user-facing view. Concurrent processing and synchronization mechanisms handle shared state safely. Figure 6.1 summarises the flow, and the following subsections describe each stage.

6.4.1 Event Detection Algorithm

The watcher waits for a supported file-system notification, identifies the affected path and event type, creates a structured event object and forwards it for processing. Error handling records failures rather than terminating the monitoring process.

```
Algorithm : EventDetection
BEGIN
  WAIT for file-system notification
  path, event_type <- IdentifyAffectedFile(notification)
  event <- BuildFileEvent(path, event_type) // path, size, times, attributes
  FORWARD event to processing pipeline
  ON error: RECORD failure in log; CONTINUE monitoring
END
```

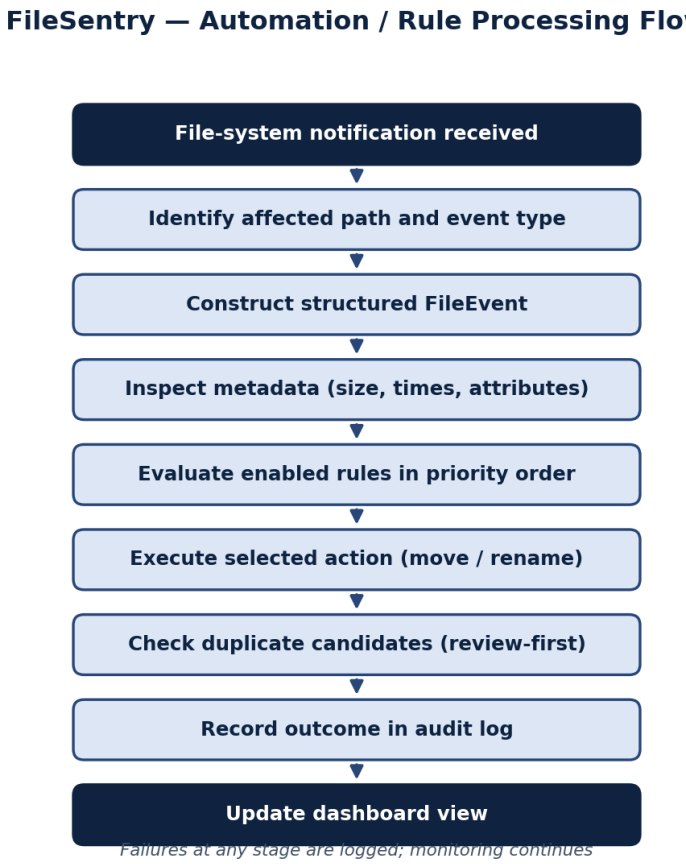


Figure 6.1: FileSentry, Automation / Rule Processing Flow

6.4.2 Rule Evaluation

Rules are evaluated against available file characteristics and configured conditions. Matching rules determine the action to be applied. A prioritized event queue can be used so that critical events are processed promptly.

```
Algorithm : RuleEvaluation
BEGIN
  INPUT event, rules
  FOR each rule in rules ORDERED BY priority
    IF rule.enabled AND rule.condition(event) THEN
      SELECT rule.action
    END IF
  END FOR
  RETURN selected action(s)
END
```

6.4.3 Duplicate Detection Flow

Candidate files are compared using relevant metadata and, where appropriate, content-derived signatures. Matching candidates are surfaced for automated or user-reviewed handling according to the configured policy.

```
Algorithm : DuplicateDetection
BEGIN
  FOR each candidate file
    IF metadata matches a known file THEN
      digest <- SHA256(contents(candidate))
      ADD candidate to duplicate group keyed by digest
    END IF
  END FOR
  SURFACE duplicate groups for review
END
```

6.4.4 Action Execution

Actions include organization operations such as moving or renaming files according to the defined rule. The action layer remains separate from event detection so that failures can be logged and retried or investigated.

6.4.5 Concurrency and Synchronization

File-system events can arrive faster than they can be fully processed, particularly during bulk operations such as extracting an archive into a monitored

folder. A prioritized event queue lets time-sensitive events be processed ahead of background activity such as duplicate scanning. Where two rules could apply to the same file, the separation of rule evaluation from action execution provides a point at which conflicts are resolved by rule priority before any file-system action is taken.

6.5 COMMUNICATION INTERFACE (API REFERENCE)

6.5.1 Monitoring Services

Monitoring-related communication carries folder state, file events and service status. It answers three recurring questions for the dashboard: which folders are registered and whether they are active or paused, which events have recently occurred, and whether the watcher and related background services are running normally.

6.5.2 Rule Services

Rule services cover retrieving the current rule set, creating or editing a rule's condition, action and priority, and toggling a rule between enabled and disabled without deleting its configuration. Evaluation itself takes place in the automation engine, so rules continue to be enforced even if the dashboard is temporarily unreachable.

6.5.3 Logging Services

Logging and telemetry communication exposes the information required for diagnosis, audit and performance monitoring. Separating log retrieval from telemetry retrieval allows the Activity Logs view to request a manageable page of recent entries without pulling the full metrics stream used for performance dashboards.

6.5.4 Dashboard Data

The dashboard consumes structured JSON describing current monitoring state, recent events and organization outcomes. Because it never touches the file system

directly, the presentation layer remains simple and largely stateless, and a new dashboard view generally needs only a new aggregation on the API side.

6.5.5 Error Handling

Communication failures, file access errors and processing exceptions are handled explicitly and recorded in persistent logs. A transient failure, such as a network interruption between the client and the dashboard or a file briefly locked by another process, is logged and retried rather than silently dropping an event or corrupting the audit trail.

The error-handling mechanism also helps distinguish between temporary failures and errors that require user attention. Temporary conditions, such as a file being accessed by another application or a directory becoming temporarily unavailable, can be handled through controlled retry attempts. If the operation continues to fail, the system should record the failure details and allow other independent file-processing tasks to continue wherever possible. This prevents a single problematic file from unnecessarily interrupting the entire monitoring workflow.

Overall, explicit error handling improves the reliability, transparency, and maintainability of FileSentry. By combining validation, controlled retries, persistent logging, and clear failure reporting, the system can respond to common operational problems while minimizing disruption to other monitoring and organization activities.

CHAPTER 7

EVALUATION METRICS

7.1 TESTING APPROACH

FileSentry is validated through automated testing, unit testing, browser testing, API testing with Postman, regression testing and user acceptance testing. Unit tests exercise rule evaluation against synthetic FileEvent objects without touching a real file system; API tests exercise the REST and JSON boundary; browser tests exercise the dashboard views; and regression testing ensures that new automation capabilities do not introduce unexpected side effects.

7.2 PERFORMANCE AND OBSERVABILITY

The architecture includes telemetry for CPU usage, memory consumption, disk I/O and network utilization, visualised through monitoring dashboards. Because the watcher is idle between notifications rather than polling, steady-state resource usage is intended to stay low. No numerical benchmark values are reported in this document.

7.3 INTEGRATION VALIDATION

Integration testing verifies communication between the desktop client, file watcher, automation engine and dashboard. A representative test creates or modifies a file in a monitored folder, confirms that the watcher emits an event, that the correct rule produces the expected action, and that the audit entry is visible through the dashboard API.

7.4 RESULT INTERPRETATION

A run is successful when an event is detected, the intended rule is applied, the file-system state reflects the action and the operation is visible in the audit trail. An action is correct only if its cause, effect and record are consistent with one another.

CHAPTER 8

SYSTEM REQUIREMENTS

8.1 SOFTWARE REQUIREMENTS

- Operating system: Windows (file-monitoring layer, Windows file-system APIs)
- Desktop / client component: .NET runtime and SDK (C#/.NET)
- Dashboard service: Python with Flask; JSON-based communication
- Database: PostgreSQL instance, local or remote
- Testing tools: Postman for API testing; automated test suite
- Observability (where deployed): Elasticsearch/Kibana, Prometheus/Grafana
- Client access: modern web browser

8.2 HARDWARE REQUIREMENTS

A development or deployment machine requires a modern CPU, sufficient RAM for the operating system and the monitoring and dashboard processes, and adequate storage for monitored data and logs. The notification-based watcher imposes a modest resource profile relative to the size of the folders observed.

8.3 NETWORK AND STORAGE CONSIDERATIONS

Local monitoring operates around the host file system, while dashboard communication and centralized observability may introduce network dependencies depending on deployment. Storage requirements grow with the number of monitored files and the retained audit records, and the log-retention period in System Settings governs the audit-trail footprint.

CHAPTER 9

RESULTS AND DISCUSSIONS

9.1 INTRODUCTION

This chapter presents ten screens of the FileSentry interface, arranged in the order an operator would typically use them: signing in, reviewing the main dashboard, managing monitored folders, inspecting file activity, configuring automation rules and the rule editor, reviewing duplicate candidates, viewing analytics and activity logs, and adjusting system settings. The screens are representative UI mockups derived from the described modules and workflow; the counts and names they show are illustrative values rather than measurements of a deployed instance.

9.2 FILESENTRY USER PORTAL

The FileSentry portal provides access to monitoring status, configured folders, activity records, automation rules, duplicate candidates, analytics, logs and settings through a single navigation panel, described in the following subsections.

The FileSentry User Portal acts as the central interface through which users interact with the Smart File Monitoring and Auto-Organization System. It brings the major functions of the application together in a structured layout, allowing users to access monitoring information, review file activities and manage automation settings without navigating between separate applications. The navigation panel provides access to the different modules of the system and helps users move between them efficiently

9.2.1 Login

The login screen establishes a session before any monitoring or configuration data is exposed. It accepts an email address and password and presents a system-status summary showing the state of the File Watcher, rule engine, database and audit log, so that an operator can confirm the system is healthy immediately after signing in.

The FileSentry login screen serves as the authentication interface for authorized users. It requires the operator to enter an email address and password before accessing monitoring information and configuration options. This authentication step helps prevent unauthorized access to system data and provides a controlled entry point to the application.

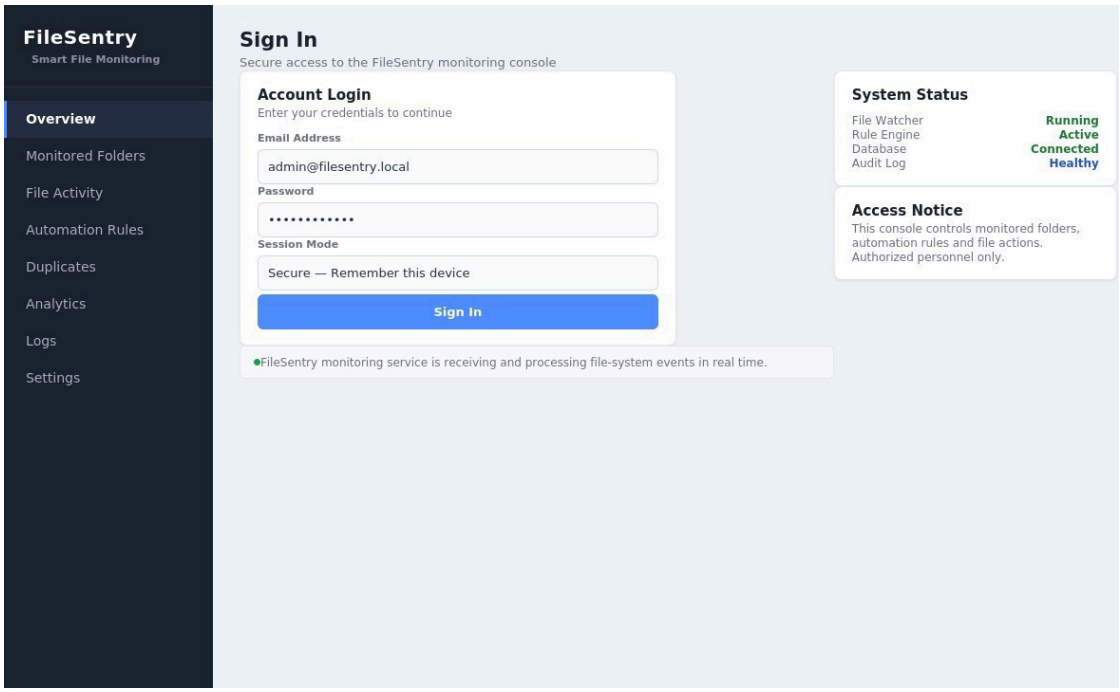


Figure 9.1: Login / Access Screen

9.2.2 Main Dashboard

The main dashboard is the operator's landing view. Summary cards present the number of monitored folders, events processed today, active rules and duplicate groups, while an activity trend chart shows recent processing load. A system-status panel reports the File Watcher, rule engine, database and audit log, and a storage panel summarises the volume under watch.

The main dashboard serves as the primary landing page of the FileSentry application, providing users with a consolidated overview of the system's current monitoring activities. It is designed to present important information in a clear and organized format, enabling users to assess the system status and access relevant details without navigating through multiple sections.

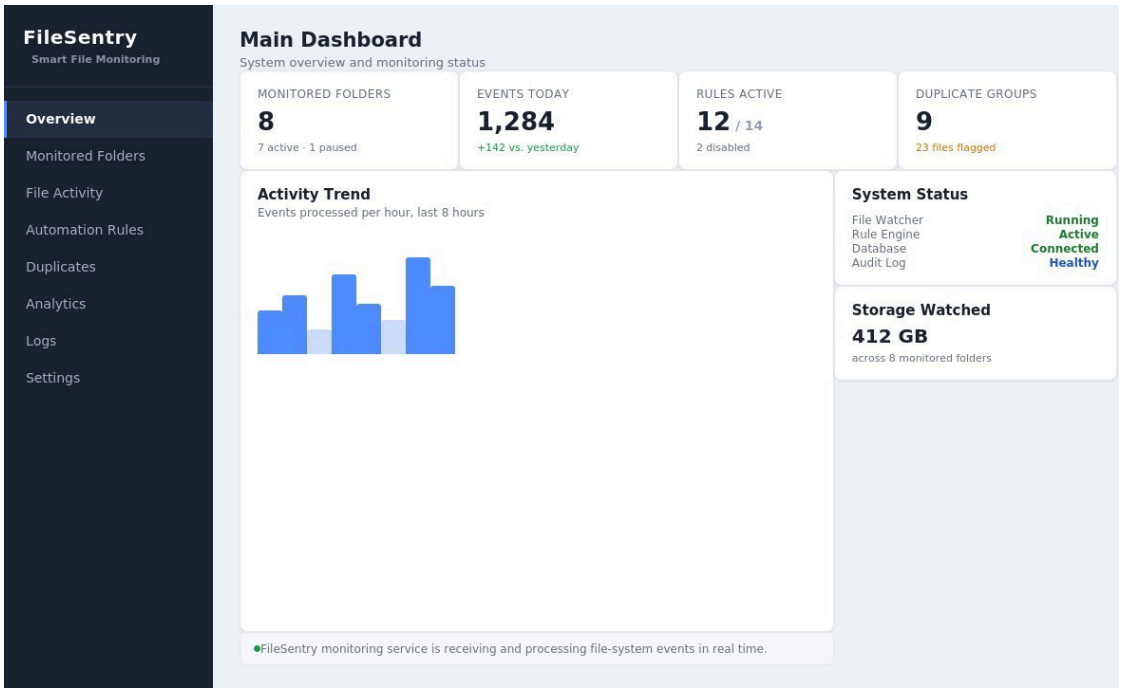


Figure 9.2: Main Dashboard, Monitoring Overview

9.2.3 Monitored Folders

The monitored-folders screen lists each configured path with its monitoring state and event count. Each path corresponds to a registration with the operating-system file-watcher API. An operator can see which folders are actively contributing events and which are paused, which is useful during bulk manual reorganization when automated actions should not interfere.

The Monitored Folders screen provides a centralized view of the directories registered with the FileSentry system. It displays each configured folder path along with its current monitoring state and associated event count. This information helps operators understand which locations are included in the monitoring process and review the activity generated by individual folders.

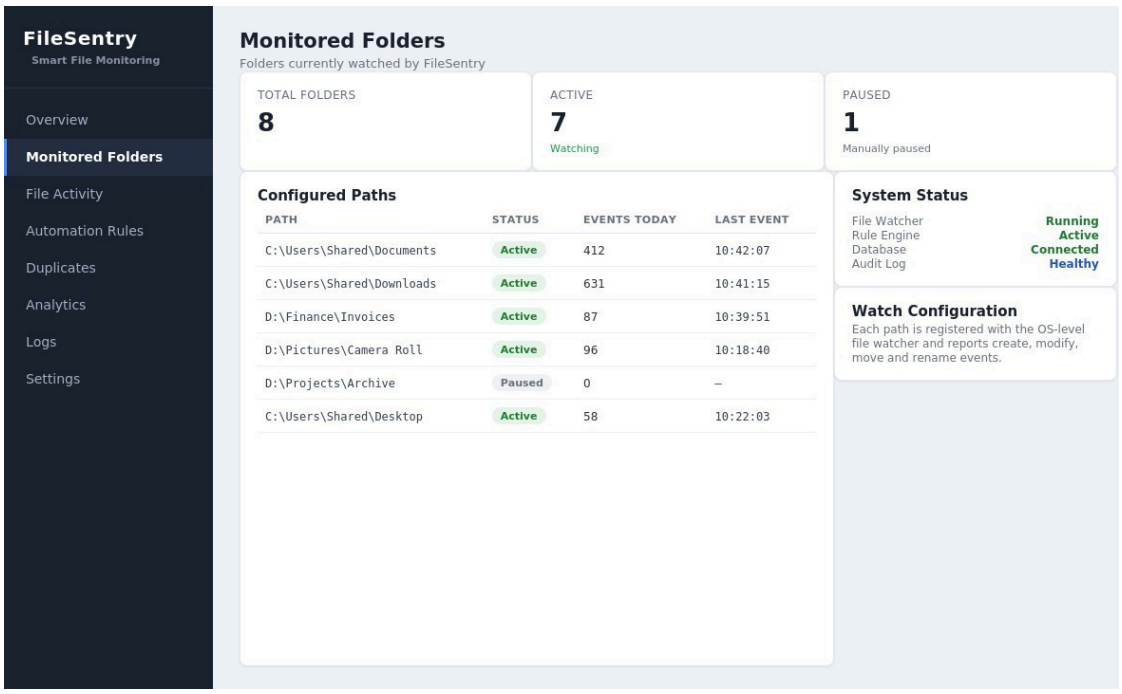


Figure 9.3: Monitored Folders, Watcher Configuration

9.2.4 File Activity

The file-activity screen lists recent create, modify, move and duplicate-related events in chronological order. Each row reflects a FileEvent: the file path, size and the type of change detected. The view makes automated activity understandable and auditable without requiring the operator to inspect the file system directly.

The File Activity screen provides a chronological record of file-related events detected by the FileSentry system. It displays recent create, modify, move and duplicate-related events, allowing operators to review changes occurring within monitored folders. By presenting these events in an organized list, the screen helps users understand file activity without manually inspecting each directory.

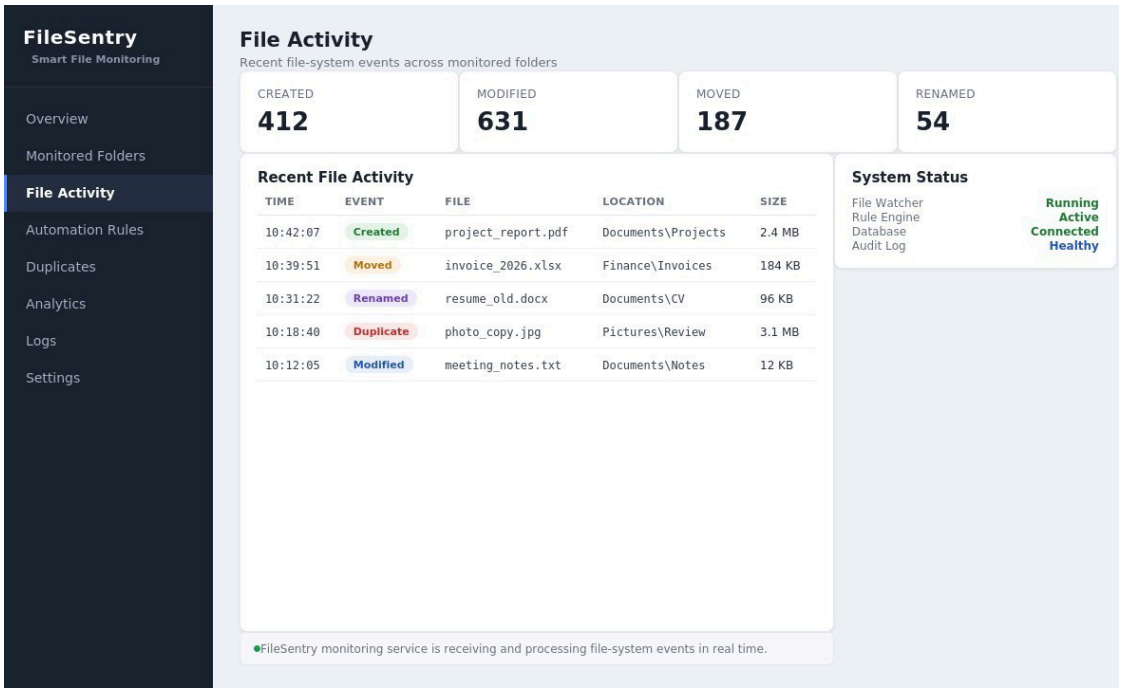


Figure 9.4: File Activity, Recent Events

9.2.5 Automation Rules

The automation-rules screen lists each rule with its condition, action, priority and enabled or disabled state, together with execution information. Presenting rules explicitly allows an operator to see why a file was moved or renamed, consistent with the traceability goal of the rule-based design.

Each rule defines the conditions under which a particular action may be performed. When a detected file event satisfies the configured condition, the rule engine evaluates the applicable rule and determines the corresponding action according to the system's execution logic. Depending on the rule configuration, an action may involve organizing a file by moving it to an appropriate location or renaming it according to a predefined pattern.

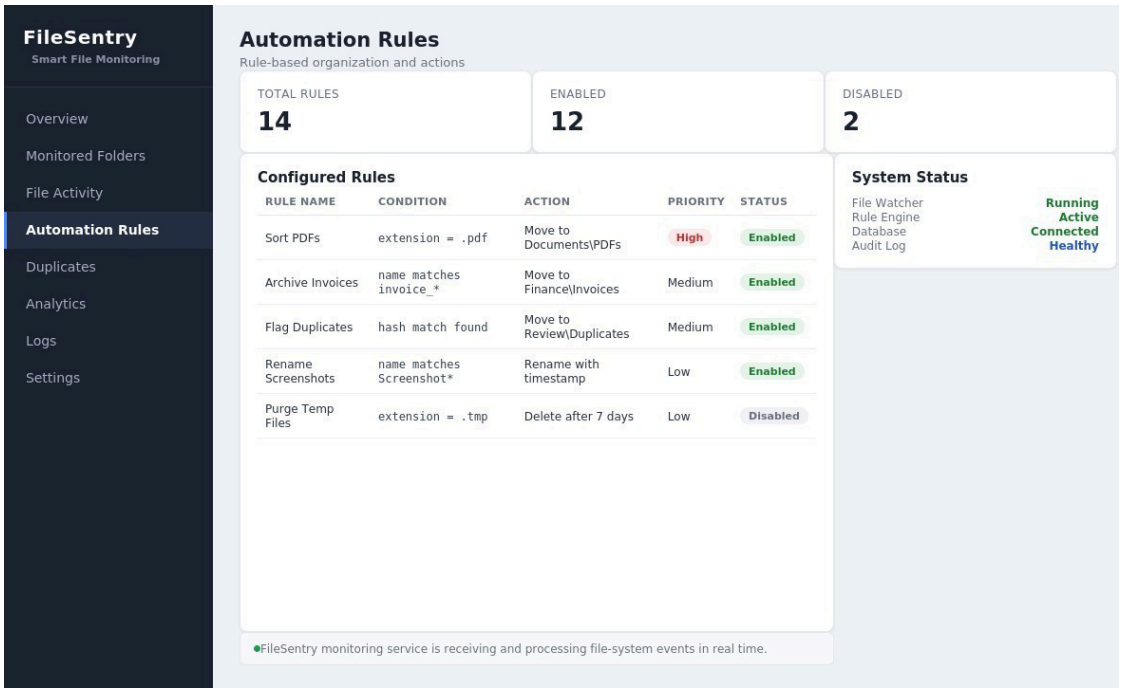


Figure 9.5: Automation Rules, Rule Status

9.2.6 Rule Editor

The rule editor separates a rule into its condition expression, its ordered actions and its priority, mirroring the structure used by the automation engine. This screen is a representative mockup of the planned visual rule editor, whose fully visual drag-and-drop form is a future enhancement and not a current capability.

The Rule Editor screen provides a structured representation of an automation rule in FileSentry. It separates the rule into three main components: the condition expression, the ordered actions and the priority. This organization reflects the structure used by the automation engine and helps users understand how a rule is defined and how it is intended to control file-management operations.

The condition expression specifies the criteria that determine when a rule may apply to a detected file event. Based on these conditions, the rule defines one or more actions to be performed when the required criteria are satisfied.

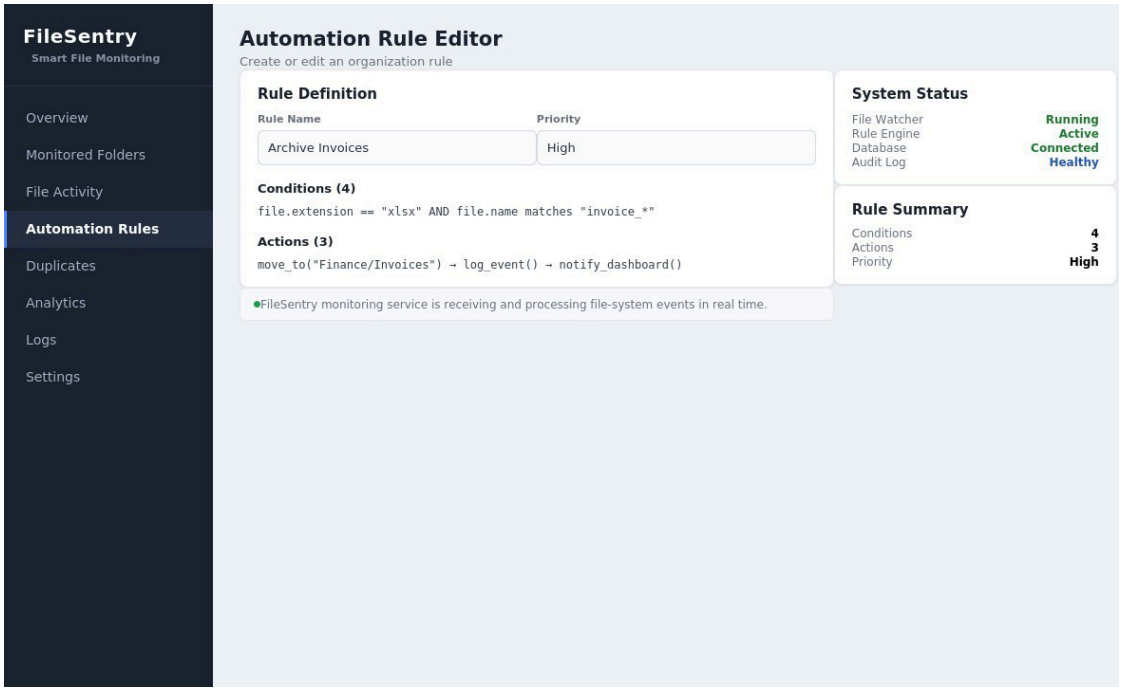


Figure 9.6: Automation Rule Editor, Conditions and Actions

9.2.7 Duplicate Detection

The duplicate-detection screen presents duplicate statistics and candidate groups, each with the basis for the match, typically a SHA-256 content hash supported by metadata comparison. The screen is built around review rather than automatic deletion, giving the operator a final decision before any file is moved or removed.

The Duplicate Detection screen provides an overview of potential duplicate files identified by the FileSentry system. It presents duplicate-related statistics and groups of candidate files, allowing operators to review files that may contain identical content. By organizing potential duplicates into groups, the interface helps users identify repeated files and understand where duplicate content may exist across monitored locations.

Each candidate group includes information about the basis for the match. A typical matching method uses the SHA-256 content hash to compare file contents, supported by metadata comparison where applicable.

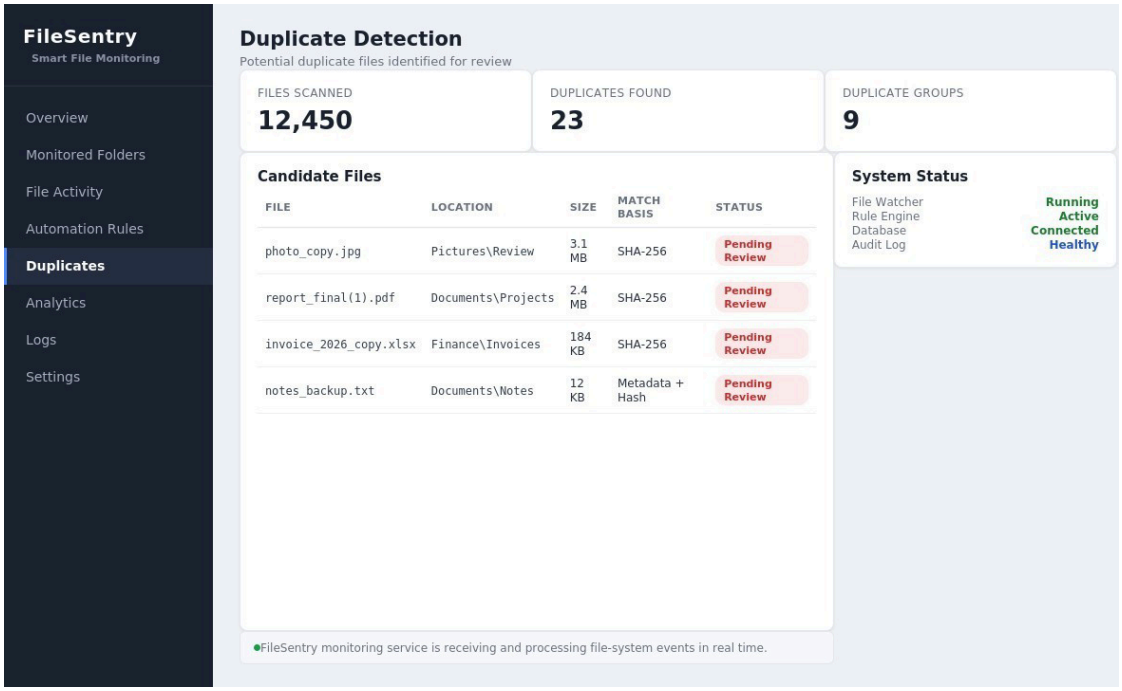


Figure 9.7: Duplicate Detection, Candidate Files

9.2.8 Analytics

The analytics screen shows monitoring and automation activity trends and distinguishes automated actions from manual ones, giving a sense of how much organizational effort the rule engine absorbs. Richer per-folder and per-rule reporting is a future enhancement.

The Analytics screen provides a summarized view of file-monitoring and automation activity within the FileSentry system. It presents activity trends in a visual format, helping operators understand how the system is being used and how file-related operations change over time. By consolidating activity information in one place, the screen makes it easier to review the overall behavior of the monitoring and automation processes.

The monitoring trends provide an overview of file events detected across the monitored locations, while automation trends highlight activity associated with rule-based operations.

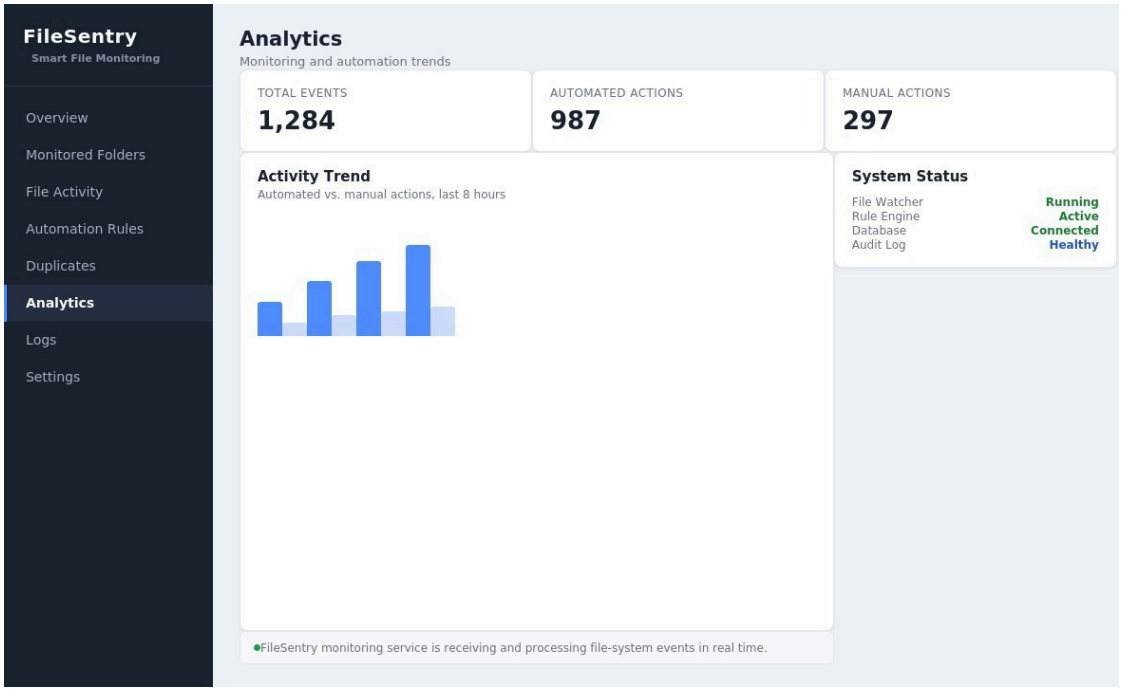


Figure 9.8: Analytics, Monitoring and Automation Trends

9.2.9 Activity Logs

The activity-logs screen presents an auditable history of events and outcomes. Unlike File Activity, which emphasises file-system events, it emphasises system-level messages: informational entries, warnings and errors raised by the watcher, automation engine or dashboard API, supporting troubleshooting and operational review.

The Activity Logs screen provides a centralized view of system-level messages generated during the operation of the FileSentry application. It presents an auditable history of recorded events and outcomes, helping operators review system behavior, identify potential issues and understand how different components perform during monitoring and automation activities.

Unlike the File Activity screen, which focuses primarily on file-system changes such as file creation, modification and movement, the Activity Logs screen emphasizes messages generated by the system itself.

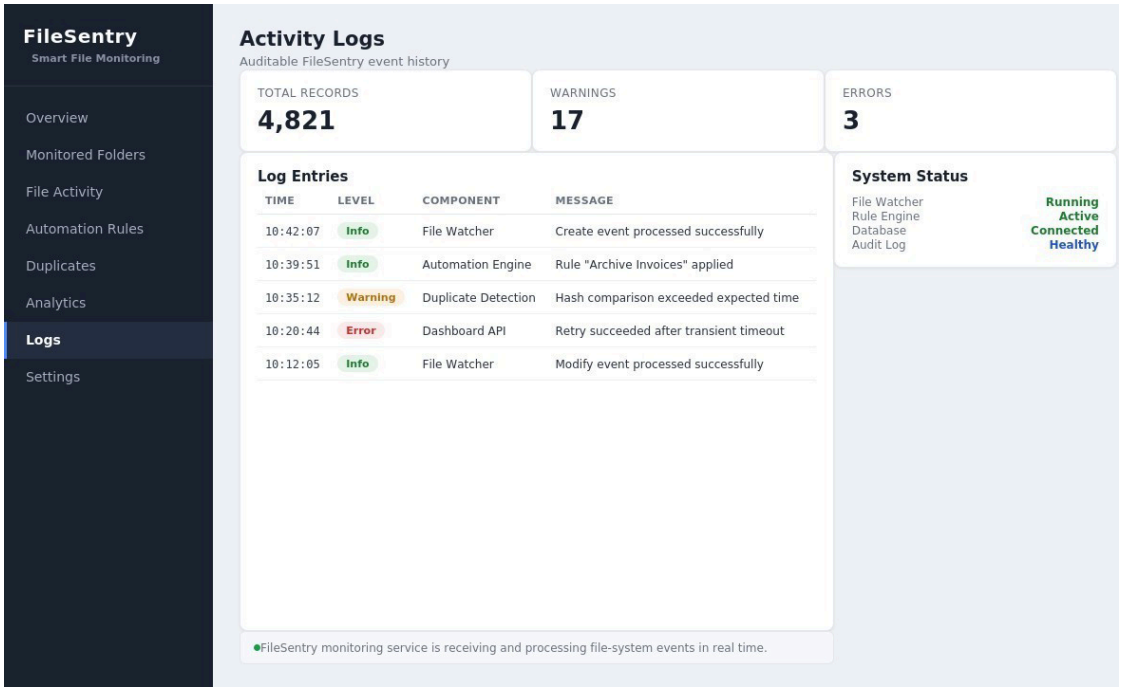


Figure 9.9: Activity Logs, Audit History

9.2.10 System Settings

The system-settings screen consolidates parameters that affect the whole system rather than a single folder or rule: the duplicate-detection method, notification behaviour, log-retention period and the database connection. Centralizing these controls keeps configuration changes auditable and separate from day-to-day monitoring.

The System Settings screen provides a centralized location for managing configuration parameters that influence the overall operation of FileSentry. Unlike folder-specific settings or individual automation rules, these options affect system-wide behavior. Bringing them together in one interface helps operators review the application's configuration and maintain a consistent approach to system management.

The duplicate-detection settings define the method used to identify potential duplicate files. Depending on the available configuration options, this may include content-based comparison using file hashes and supporting metadata.

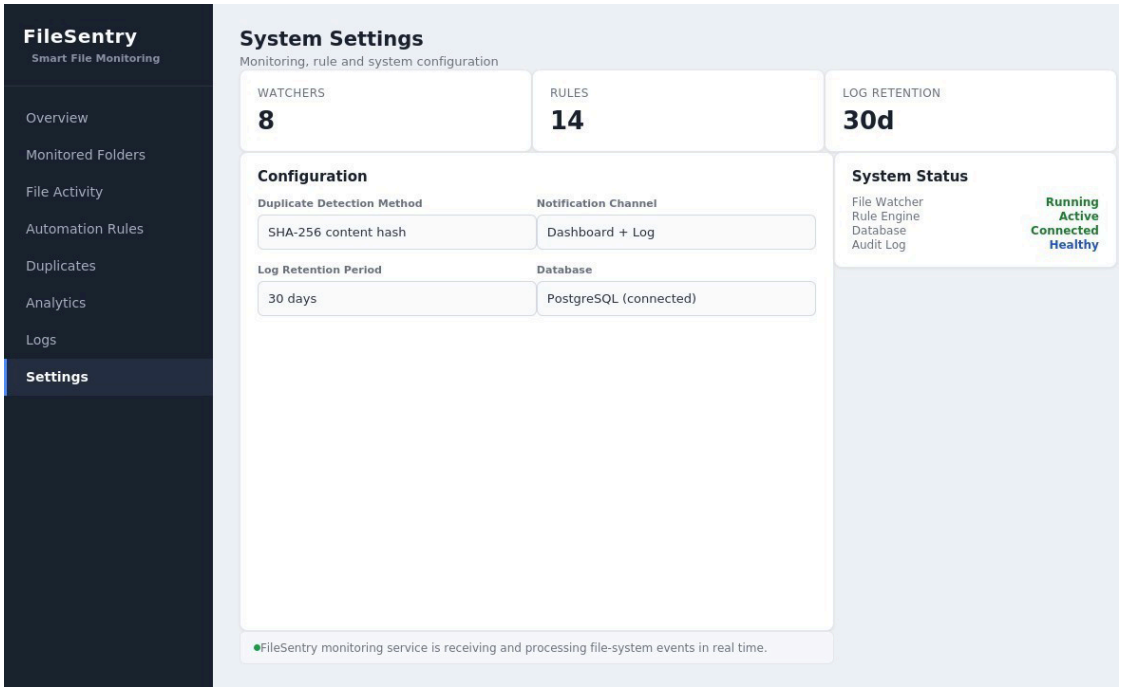


Figure 9.10: System Settings, Monitoring Configuration

CHAPTER 10

CONCLUSION AND FUTURE WORK

10.1 CONCLUSION

FileSentry provides a modular approach to real-time file monitoring and automated organization. The architecture separates file-system observation from metadata processing, rule evaluation, action execution and presentation. This separation supports maintainability, logging and future extension. The dashboard adds a human-readable operational layer, while the audit trail gives visibility into automated actions, forming a foundation for evolving from basic file monitoring toward a broader intelligent organization platform.

The current design separates detection, processing, rules, actions and observability, which is important because future capabilities can be introduced without forcing all components to change at once. The explicit, rule-based approach keeps every automated action explainable.

10.2 FUTURE ENHANCEMENT

The following directions are proposed future enhancements and are not part of the current implementation: integration with Google Drive, Dropbox and OneDrive; a visual drag-and-drop rule editor with richer conditions; advanced reporting; stronger role-based access control; distributed processing with asynchronous queues; cross-platform Electron deployment; semantic search using natural language; and user-behaviour analysis for adaptive rule prioritization. Each should be evaluated incrementally with attention to security, data integrity, user control and operational reliability.

REFERENCES

[1] Python Software Foundation. (2024). Python 3 Documentation.

[2] Python Software Foundation. (2024). os — Miscellaneous Operating System Interfaces. Python Documentation.

[3] Python Software Foundation. (2024). shutil — High-Level File Operations. Python Documentation.

[4] Python Software Foundation. (2024). hashlib — Secure Hashes and Message Digests. Python Documentation.

[5] Python Software Foundation. (2024). logging — Logging Facility for Python. Python Documentation.

[6] Python Software Foundation. (2024). sqlite3 — DB-API 2.0 Interface for SQLite Databases. Python Documentation.

[7] Bares, A. (n.d.). Watchdog: Python API Library and Shell Utilities to Monitor File System Events. Python Package Index.

[8] National Institute of Standards and Technology. (2015). Secure Hash Standard (SHS) (FIPS PUB 180-4).

[9] Pallets. (2024). Werkzeug Documentation.

[10] Python Software Foundation. (2024). unittest — Unit Testing Framework. Python Documentation.

[11] Python Software Foundation. (2024). argparse — Parser for Command-Line Options, Arguments and Subcommands. Python Documentation.

[12] Python Packaging Authority. (n.d.). Python Packaging User Guide.

APPENDIX

KEY SOURCE CODE

The following pages present representative, condensed excerpts from the key source files of FileSentry, illustrative of the module design described in the previous chapters. Each heading names the component being illustrated; ellipsis comments mark omitted boilerplate or repeated patterns. Secret values are shown as [REDACTED]. The listings are illustrative restatements of module behaviour, not verbatim extracts of deployed source files.

A.1 FileEvent Structure

```
FileEvent {
    Path           : text        // full file-system location
    EventType      : enum        // Create | Modify | Move | Rename
    Size           : integer     // file size at the time of the event
    ModifiedTime   : timestamp   // most recent content change
    LastAccessTime : timestamp   // most recent file access
    Attributes     : flags       // e.g. read-only, hidden
}
```

A.2 File Watcher (C#)

```
public sealed class FolderWatcher : IDisposable
{
    private readonly FileSystemWatcher _watcher;
    private readonly IEventSink _sink;
    private readonly ILogger _log;

    public FolderWatcher(string path, IEventSink sink, ILogger log)
    {
        _sink = sink;
        _log = log;
        _watcher = new FileSystemWatcher(path)
        {
            IncludeSubdirectories = true,
            NotifyFilter = NotifyFilters.FileName | NotifyFilters.LastWrite
                         | NotifyFilters.Size | NotifyFilters.Attributes,
            InternalBufferSize = 64 * 1024     // absorb bursts of events
        };
        _watcher.Created += (s, e) => Forward(e.FullPath, EventKind.Create);
        _watcher.Changed += (s, e) => Forward(e.FullPath, EventKind.Modify);
        _watcher.Renamed += (s, e) => Forward(e.FullPath, EventKind.Rename);
        _watcher.Error   += (s, e) => _log.Error("Watcher error", e.GetException());
    }

    public void Start() => _watcher.EnableRaisingEvents = true;
    public void Pause() => _watcher.EnableRaisingEvents = false;

    private void Forward(string path, EventKind kind)
```

```
    {
        try { _sink.Enqueue(MetadataProcessor.Build(path, kind)); }
        catch (IOException ex)     // locked or inaccessible: record, keep going
        {
            _log.Warn($"Skipped {path}: {ex.Message}");
        }
    }

    public void Dispose() => _watcher.Dispose();
}
```

A.3 Metadata Processor (C#)

```
public static class MetadataProcessor
{
    public static FileEvent Build(string path, EventKind kind)
    {
        var info = new FileInfo(path);
        return new FileEvent
        {
            Path           = info.FullName,
            EventType      = kind,
            Size           = info.Exists ? info.Length : 0,
            ModifiedTime   = info.LastWriteTimeUtc,
            LastAccessTime = info.LastAccessTimeUtc,
            Attributes     = info.Attributes
        };
    }
}
```

A.4 Prioritized Event Queue (C#)

```
public sealed class EventQueue : IEventSink
{
    private readonly PriorityQueue<FileEvent, int> _queue = new();
    private readonly SemaphoreSlim _signal = new(0);
    private readonly object _gate = new();

    public void Enqueue(FileEvent e)
    {
        // 0 = time-sensitive; 9 = background work such as duplicate scans
        int priority = e.IsBackground ? 9 : 0;
        lock (_gate) { _queue.Enqueue(e, priority); }
        _signal.Release();
    }

    public async Task<FileEvent> DequeueAsync(CancellationToken ct)
    {
        await _signal.WaitAsync(ct);
        lock (_gate) { return _queue.Dequeue(); }
    }




}
```

A.5 Rule Model and Automation Engine (C#)

```
public sealed class Rule
{
    public string Name { get; init; }
    public int Priority { get; init; }          // 1 = highest
    public bool Enabled { get; init; }
    public Func<FileEvent, bool> Condition { get; init; }
    public IFileAction Action { get; init; }
}

public sealed class AutomationEngine
{
    private readonly IReadOnlyList<Rule> _rules;
    private readonly bool _firstMatchOnly;

    public AutomationEngine(IReadOnlyList<Rule> rules, bool firstMatchOnly)
    {
        _rules = rules;
        _firstMatchOnly = firstMatchOnly;
    }

    public IReadOnlyList<Rule> Evaluate(FileEvent e)
    {
        var selected = new List<Rule>();
        foreach (var rule in _rules.OrderBy(r => r.Priority))
        {
            if (rule.Enabled && rule.Condition(e))
            {
                selected.Add(rule);
                if (_firstMatchOnly) break;
            }
        }
        return selected;
    }
}

// Example rule: "Sort PDFs" (extension = .pdf -> Documents\PDFs)
var sortPdfs = new Rule {
    Name = "Sort PDFs", Priority = 1, Enabled = true,
    Condition = e => Path.GetExtension(e.Path)
                         .Equals(".pdf", StringComparison.OrdinalIgnoreCase),
    Action = new MoveAction(@"Documents\PDFs") };
```

A.6 Action Execution (C#)

```
public sealed class MoveAction : IFileAction
{
    private readonly string _targetFolder;
    public MoveAction(string targetFolder) => _targetFolder = targetFolder;

    public ActionResult Execute(FileEvent e)
    {
        Directory.CreateDirectory(_targetFolder);
        string target = Path.Combine(_targetFolder, Path.GetFileName(e.Path));
        if (File.Exists(target)) target = UniqueName(target);  // never overwrite
        File.Move(e.Path, target);
        return ActionResult.Ok($"Moved to {target}");
    }

    private static string UniqueName(string path)
    {
        string dir = Path.GetDirectoryName(path);
```

```
        string stem = Path.GetFileNameWithoutExtension(path);
        string ext = Path.GetExtension(path);
        for (int i = 1; ; i++)
        {
            string candidate = Path.Combine(dir, $"{stem} ({i}){ext}");
            if (!File.Exists(candidate)) return candidate;
        }
    }
}

public sealed class ActionExecutor
{
    public void Run(FileEvent e, IEnumerable<Rule> rules)
    {
        foreach (var rule in rules)
        {
            try
            {
                var result = rule.Action.Execute(e);
                _audit.Info(e, rule, result.Message);
            }
            catch (IOException ex)      // file briefly locked: log and retry
            {
                _audit.Warn(e, rule, ex.Message);
                _retry.Schedule(e, rule, TimeSpan.FromSeconds(5));
            }
            catch (Exception ex)
            {
                _audit.Error(e, rule, ex.Message);
            }
        }
    }
}
```

A.7 Duplicate Detection with SHA-256 (C#)

```
public sealed class DuplicateDetector
{
    private readonly Dictionary<long, List<string>> _bySize = new();
    private readonly Dictionary<string, string> _digests = new();
    private readonly Dictionary<string, HashSet<string>> _groups = new();

    public void Observe(FileEvent e)
    {
        if (!_bySize.TryGetValue(e.Size, out var peers))
            _bySize[e.Size] = peers = new List<string>();

        if (peers.Count > 0)                    // metadata matches a known file
        {
            foreach (var path in peers.Append(e.Path))
            {
                string digest = DigestOf(path);
                if (!_groups.TryGetValue(digest, out var group))
                    _groups[digest] = group = new HashSet<string>();
                group.Add(path);
            }
        }
        peers.Add(e.Path);
    }

    // Candidates are surfaced for review; nothing is deleted automatically.
    public IEnumerable<IReadOnlyCollection<string>> GroupsForReview() =>
        _groups.Values.Where(g => g.Count > 1);
```

```
    private string DigestOf(string path)
    {
        if (_digests.TryGetValue(path, out var cached)) return cached;
        using var sha = SHA256.Create();
        using var stream = File.OpenRead(path);
        return _digests[path] = Convert.ToHexString(sha.ComputeHash(stream));
    }
}
```

A.8 Event Publisher over REST and JSON (C#)

```
public sealed class DashboardClient
{
    private readonly HttpClient _http;
    public DashboardClient(HttpClient http) => _http = http;

    public async Task PublishAsync(FileEvent e, CancellationToken ct)
    {
        var json = JsonSerializer.Serialize(new
        {
            path = e.Path,
            event_type = e.EventType.ToString(),
            size = e.Size,
            modified_time = e.ModifiedTime,
            last_access_time = e.LastAccessTime
        });
        using var body = new StringContent(json, Encoding.UTF8, "application/json");
        var response = await _http.PostAsync("/api/events/", body, ct);
        response.EnsureSuccessStatusCode();   // failures are logged and retried
    }
}
```

A.9 Flask Application Factory (Python)

```
from flask import Flask, jsonify
from flask_cors import CORS
from config import Config
from models import db


def create_app(config_class=Config):
    app = Flask(__name__)
    app.config.from_object(config_class)
    CORS(app)
    db.init_app(app)

    from routes.folders import folders_bp
    from routes.events import events_bp
    from routes.rules import rules_bp
    # ... remaining blueprints: duplicates, logs, analytics, settings
    app.register_blueprint(folders_bp, url_prefix='/api/folders')
    app.register_blueprint(events_bp, url_prefix='/api/events')
    app.register_blueprint(rules_bp, url_prefix='/api/rules')
    # ... remaining blueprints registered the same way

    @app.route('/api/health')
    def health_check():
        return jsonify({"status": "healthy"}), 200

    return app
```

A.10 Configuration (Python)

```
import os


class Config:
    SECRET_KEY = os.environ.get('SECRET_KEY') or '[REDACTED]'
    SQLALCHEMY_DATABASE_URI = os.environ.get('DATABASE_URL') or \
        'postgresql://filesentry:[REDACTED]@localhost:5432/filesentry'
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    LOG_RETENTION_DAYS = 30            # shown on the System Settings screen
    DUPLICATE_METHOD = 'sha256'
    NOTIFICATION_CHANNEL = 'dashboard+log'
```

A.11 Database Models (Python)

```
class MonitoredFolder(db.Model):
    __tablename__ = 'monitored_folders'
    id = db.Column(db.Integer, primary_key=True)
    path = db.Column(db.String(400), unique=True, nullable=False)
    state = db.Column(db.String(10), default='active')    # active | paused


class FileEventRecord(db.Model):
    __tablename__ = 'file_events'
    id = db.Column(db.Integer, primary_key=True)
    folder_id = db.Column(db.Integer, db.ForeignKey('monitored_folders.id'))
    path = db.Column(db.String(400), nullable=False)
    event_type = db.Column(db.String(10), nullable=False)
    size = db.Column(db.BigInteger)
    modified_time = db.Column(db.DateTime)
    last_access_time = db.Column(db.DateTime)
    recorded_at = db.Column(db.DateTime, default=datetime.utcnow)


class AutomationRule(db.Model):
    __tablename__ = 'automation_rules'
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(100), nullable=False)
    condition = db.Column(db.Text, nullable=False)
    action = db.Column(db.String(200), nullable=False)
    priority = db.Column(db.Integer, default=2)
    enabled = db.Column(db.Boolean, default=True)


class AuditLog(db.Model):
    __tablename__ = 'audit_logs'
    id = db.Column(db.Integer, primary_key=True)
    event_id = db.Column(db.Integer, db.ForeignKey('file_events.id'))
    rule_id = db.Column(db.Integer, db.ForeignKey('automation_rules.id'))
    level = db.Column(db.String(10), nullable=False)   # info | warning | error
    component = db.Column(db.String(40), nullable=False)
    message = db.Column(db.Text, nullable=False)
    created_at = db.Column(db.DateTime, default=datetime.utcnow)
```

A.12 Event Ingestion and Rule Routes (Python)

```
@events_bp.route('/', methods=['POST'])
def ingest_event():
    data = request.get_json(silent=True) or {}
    if not data.get('path') or not data.get('event_type'):
        return jsonify(msg="path and event_type are required"), 400
    record = FileEventRecord(path=data['path'],
                             event_type=data['event_type'],
                             size=data.get('size'))
    db.session.add(record)
    db.session.commit()
    return jsonify({"id": record.id}), 201


@rules_bp.route('/', methods=['GET'])
def list_rules():
    rules = AutomationRule.query.order_by(AutomationRule.priority).all()
    return jsonify([r.to_dict() for r in rules]), 200


@rules_bp.route('/<int:rule_id>/toggle', methods=['PATCH'])
def toggle_rule(rule_id):
    rule = AutomationRule.query.get_or_404(rule_id)
    rule.enabled = not rule.enabled        # configuration is kept, not deleted
    db.session.commit()
    return jsonify(rule.to_dict()), 200
```

A.13 Activity Log Route (Python)

```
@logs_bp.route('/', methods=['GET'])
def recent_logs():
    page = request.args.get('page', 1, type=int)
    level = request.args.get('level')      # info | warning | error
    query = AuditLog.query.order_by(AuditLog.created_at.desc())
    if level:
        query = query.filter_by(level=level)
    items = query.paginate(page=page, per_page=50, error_out=False).items
    return jsonify([{"time": i.created_at.isoformat(),
                     "level": i.level,
                     "component": i.component,
                     "message": i.message} for i in items]), 200
```

A.14 Dependencies

```
# Dashboard service (requirements.txt)
Flask==3.0.0
Flask-SQLAlchemy==3.1.1
Flask-Cors==4.0.0
SQLAlchemy==2.0.23
psycopg2-binary==2.9.9
python-dotenv==1.0.0
pytest==7.4.3

# Watcher / client (NuGet packages)
Serilog
Serilog.Sinks.File
Microsoft.Extensions.Http
prometheus-net
xunit
```

A.15 Representative Test Cases

```
// xUnit: rule evaluation against a synthetic FileEvent (no real file system)
[Fact]
public void HigherPriorityRuleIsSelectedFirst()
{
    var e = new FileEvent { Path = @"C:\Shared\Downloads\invoice_2026.pdf" };
    var engine = new AutomationEngine(TestRules.All, firstMatchOnly: true);
    var selected = engine.Evaluate(e);
    Assert.Single(selected);
    Assert.Equal("Sort PDFs", selected[0].Name);
}

# pytest: REST and JSON boundary
def test_ingest_event(client):
    payload = {"path": "C:/Shared/Documents/report.pdf",
               "event_type": "Create", "size": 2457600}
    res = client.post('/api/events/', json=payload)
    assert res.status_code == 201

def test_toggle_rule_keeps_configuration(client):
    res = client.patch('/api/rules/1/toggle')
    assert res.status_code == 200
    assert res.get_json()["enabled"] in (True, False)
```